



Data Visualization

LABORATORIO

Capitolo 3 – Manipolazione del DOM, introduzione D3



Argomenti del giorno

- **Concetti base di D3**

- Select
- Append
- Remove
- Edit

- **Databinding**

Cos'è D3?

D3.js (Data-Driven Documents) è una libreria JavaScript potente e flessibile per la visualizzazione dei dati, utilizzata per creare grafici interattivi, mappe, diagrammi e altre rappresentazioni visive direttamente nel browser.

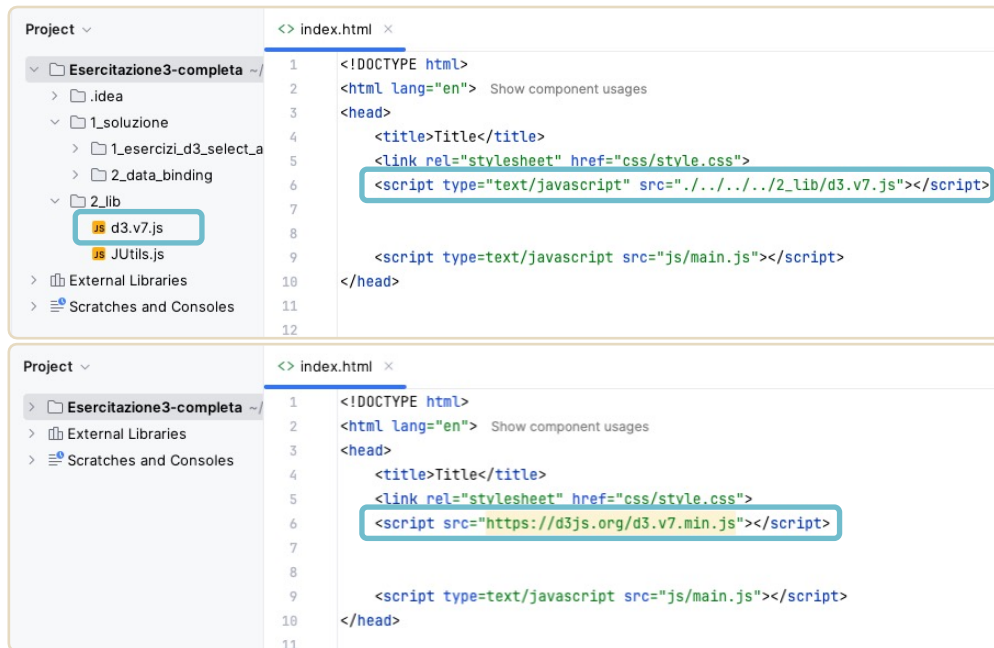
Permette di **collegare i dati a elementi HTML, SVG o Canvas** e manipolarli dinamicamente, sfruttando le capacità di **JavaScript, CSS e SVG** senza la necessità di librerie grafiche aggiuntive.



Aggiungere D3 a un progetto

Per utilizzare la libreria D3.js nel proprio progetto, è necessario aggiungerla in uno dei seguenti modi:

1. **Scaricare i file della libreria** direttamente dalla pagina ufficiale di [D3](#) e includerli nel progetto e collegarli nella sezione < head > della pagina HTML. Permette di lavorare offline e garantisce maggiore stabilità, evitando aggiornamenti automatici.
2. **Utilizzare un CDN (Content Delivery Network)** per includere D3 direttamente nella pagina HTML. Questa soluzione non richiede il download di file e assicura l'accesso alla versione più recente della libreria ma richiede una connessione a internet.



Selezionare elementi in D3

D3.js fornisce funzioni proprie che svolgono compiti simili a funzioni native di **JavaScript**, ma con un approccio più orientato alla gestione dei dati e alla manipolazione del DOM in modo dichiarativo.

Metodi come **document.querySelector()** o **document.getElementById()**, in D3 diventano:

- **d3.select()** → Seleziona il primo elemento che corrisponde a un selettore CSS
- **d3.selectAll()** → Seleziona tutti gli elementi corrispondenti e restituisce una selezione multipla.

index.html

```
<g id="mainContainer">
  <svg width="300" height="200">
    <rect x="125" y="75" width="100" height="50"></rect>
  </svg>

  <svg width="500" height="200">
    <circle r="50" cx="150" cy="100"></circle>
    <circle r="50" cx="270" cy="100"></circle>
  </svg>
</g>
```

main.js

```
// con js puro
document.querySelector("p").style.color = "red";
document.getElementById("mainContainer");

// Con D3.js
d3.select("p").style("color", "red");
d3.select("mainContainer");
d3.selectAll("circle");
```

Modifica di un elemento

D3 usa **.attr()** per impostare o ottenere un attributo HTML/SVG, invece di `setAttribute()` e `getAttribute()`.

D3.attr() prende in input

- il **nome** dell'attributo che si vuole modificare
- il **valore** da assegnare all'attributo

Se si vuole assegnare un **valore dinamico**, è possibile passare come valore una **funzione callback**.

main.js

```
//con js puro
document.querySelector("rect").setAttribute("height", 50);
document.querySelector("rect").setAttribute("width", 100);

//con D3
d3.select("rect").attr("width", 100).attr("height", 50);

d3.selectAll("circle").attr("r",
  function(_, idx){
    return 10 * (idx + 1);
  });

//con js puro
let width = document.querySelector("rect").getAttribute("width");

//con D3
let width = d3.select("rect").attr("width")
console.log("Il rettangolo ha come larghezza ", width);
```

Modifica di un elemento

D3 usa **.style()** per impostare le proprietà CSS, mentre in JS puro si usa la proprietà **.style**.

Il metodo **.text()** invece viene utilizzato per cambiare il testo all'interno di un elemento.

main.js

```
//con js puro
document.querySelector("p").style.fontSize = "20px";
document.querySelector("p").style.color = "blue";

//con D3
d3.select("p")
  .style("font-size", "20px")
  .style("color", "blue");

//con js puro
document.querySelector("p").textContent =
  "Con JS puro ho cambiato il testo del mio paragrafo!"

//con D3
d3.select("p")
  .text("Con D3 ho cambiato il testo del mio paragrafo!");
```

Eventi in D3

La funzione `.on()` viene utilizzata per associare eventi agli elementi del DOM, come click, hover, keypress, ecc.

Come `addEventListener()` di JS, prende in input il nome dell'evento e la funzione callback da eseguire quando l'evento si verifica

D3.on() si **integra meglio** con la gestione dei dati delle selezioni multiple di D3.

Generalmente, quando si usa D3, è sempre meglio utilizzare le sue funzioni libreria invece che quelle fornite da JS puro.

main.js

```
//con js puro
let rect = document.querySelector("rect");

rect.addEventListener("mouseover", () => {
  rect.setAttribute("fill", "blue");
})

rect.addEventListener("mouseleave", () => {
  rect.setAttribute("fill", "black");
})

//con D3
let rect = d3.select("rect")
rect.on("mouseover", () => rect.attr("fill", "blue"))
  .on("mouseleave", () => rect.attr("fill", "black"));
```

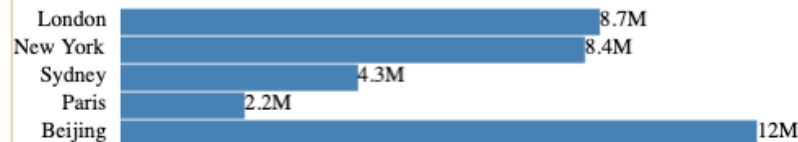

Databinding

Il **databinding** è il meccanismo che **collega i dati** a elementi HTML o SVG, ed è il cuore della **creazione di grafici** in D3: permette di **generare, aggiornare e rimuovere** elementi in base ai **dati**, rendendo le visualizzazioni **dinamiche e interattive**.

D3 segue il **paradigma Data-Driven**, ovvero gli elementi del DOM vengono **creati, aggiornati e/o rimossi automaticamente** in base ai dati forniti.

Usiamo il codice base databinding...

```
15 const data : [{name: string, population: nu...} = [  
16   {name: 'London', population: 8674000},  
17   {name: 'New York', population: 8406000},  
18   {name: 'Sydney', population: 4293000},  
19   {name: 'Paris', population: 2244000},  
20   {name: 'Beijing', population: 11510000}]
```



Concetti base databinding

Per **collegare** dati agli elementi, si usano i metodi:

- **.selectAll('element')** (Selezione) →
Seleziona un gruppo di elementi (anche se non esistono ancora)
- **.data(data)** (Associazione) →
associa un array di dati agli elementi selezionati.
- **.join('element')** (Rappresentazione) →
→ Crea nuovi elementi per ogni dato mancante.

Cosa succede nell'esempio?

1. D3 cerca **<circle>** esistenti e non ne trova.
2. Associa i dati **[75, 150, 225]**.
3. Crea tre **<circle>** con raggio **20**
4. Li posiziona su **cx = 75, 150, 225**

main.js

```
const data = [75, 150, 225];

d3.select("#svgcenter")
  .selectAll("circle") //seleziona elementi circle
  .data(data)           //associa dei dati
  .join("circle")       //crea un elemento circle per ogni dato
  .attr("cx", (d) => d)  //coordinata x del centro
  .attr("cy", 150)      //coordinata y del centro
  .attr("r", 20)         //raggio
  .attr("fill", "blue") //colore
```

prima

Proviamo la fase di ENTER

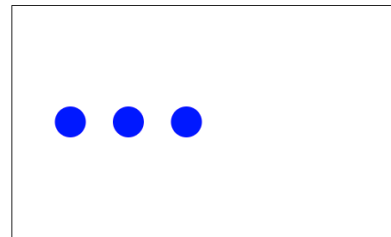
Originariamente abbiamo un riquadro vuoto. Dopo il data binding...



dopo

Proviamo la fase di ENTER

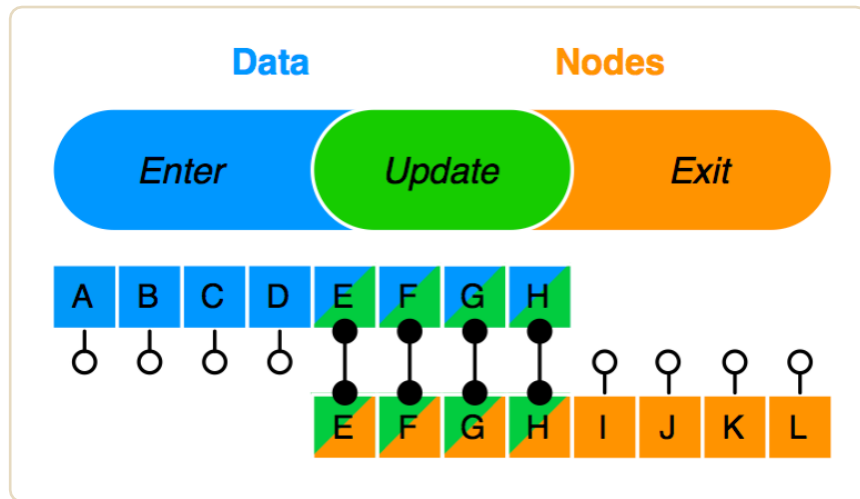
Originariamente abbiamo un riquadro vuoto. Dopo il data binding...



Le tre fasi del databinding

Quando si associano dei dati a elementi del DOM con D3, si possono verificare **tre casi**:

1. **Creazione (enter)** → Se ci sono più dati degli elementi esistenti, **vengono aggiunti nuovi elementi**.
Quindi usa `.enter.append()`.
2. **Modifica (update)** → Se il numero degli elementi è **uguale ai dati**, allora vengono semplicemente **modificati**.
3. **Rimozione (exit)** → Se ci sono **più elementi che dati**, quelli in eccesso vengono **rimossi** con `.exit.remove()`.



Da v6 si può usare `data().join()` per tutte le fasi

Aggiungere elementi in D3

Il metodo `.append()` consente di creare nuovi elementi all'interno di un contenitore.

Mentre se si vuole aggiungere **dinamicamente** più elementi si utilizzano `.data()` e `.join()` oppure `.enter.append()`.

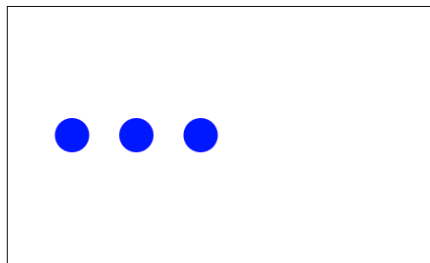
```
const data = [75, 150, 225, 300, 375];

d3.select("#svgupdate")
  .selectAll("circle") //seleziona elementi circle
  .data(data)           //associa i dati
  .join("circle")       //crea un elemento circle per ogni dato se non esiste
  .attr("cx", (d) => d) //coordinata x del centro
  .attr("cy", 150)      //coordinata y del centro
  .attr("r", 20)        //raggio
  .attr("fill", "blue") //colore
}
```

prima

Proviamo la fase di UPDATE

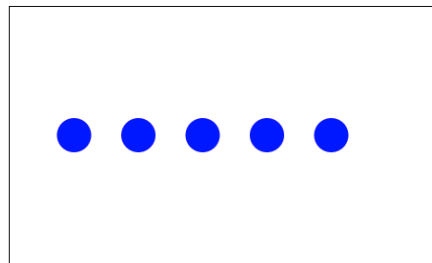
Originariamente abbiamo un riquadro con 3 cerchi. Dopo il data binding...



dopo

Proviamo la fase di UPDATE

Originariamente abbiamo un riquadro con 3 cerchi. Dopo il data binding...



Rimuovere elementi in D3

Analogamente, se si vuole rimuovere un elemento si utilizza `.remove()`.

Mentre per rimuovere elementi creati con `.data()`, si utilizza la combinazione `.exit().remove()` che permette di rimuovere elementi in eccesso. Oppure direttamente `.data()` e `.join()`

Codice equivalente con `exit().remove()`

```
const data = [75, 150];

d3.select("#svgremove")
  .selectAll("circle") //seleziona elementi circle
  .data(data)           //associa gli elementi
  .attr("cx", (d) => d) //coordinata x del centro
  .attr("cy", 150)      //coordinata y del centro
  .attr("r", 20)         //raggio
  .attr("fill", "blue")  //colore
  .exit().remove()
```

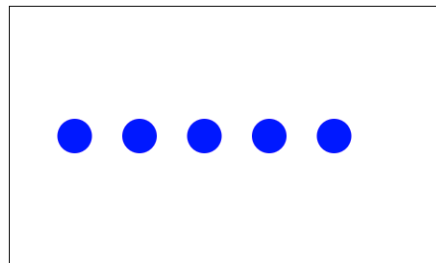
```
const data = [75, 150];

d3.select("#svgremove")
  .selectAll("circle") //seleziona elementi circle
  .data(data)           //associa gli elementi
  .join("circle")       //crea un elemento circle per ogni dato
  .attr("cx", (d) => d) //coordinata x del centro
  .attr("cy", 150)      //coordinata y del centro
  .attr("r", 20)         //raggio
  .attr("fill", "blue")  //colore
```

prima

Proviamo la fase di REMOVE

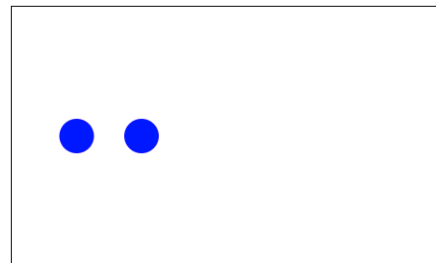
Originariamente abbiamo un riquadro con 5 cerchi. Dopo il data binding...



dopo

Proviamo la fase di REMOVE

Originariamente abbiamo un riquadro con 5 cerchi. Dopo il data binding...





Esercizi

Svolgere gli esercizi presenti nella sezione Esercitazione 3.

L'esercitazione è divisa in due sottocartelle principali.

La prima è relativa ai concetti base di D3, quindi come aggiungere, modificare ed eliminare elementi.

La seconda prevede esercizi sul databinding, con i primi accenni alla creazione di grafici.

